系统硬件设计-汇报展示

指令架构：RISCV

汇报人：202221219-李易简

五级流水线设计

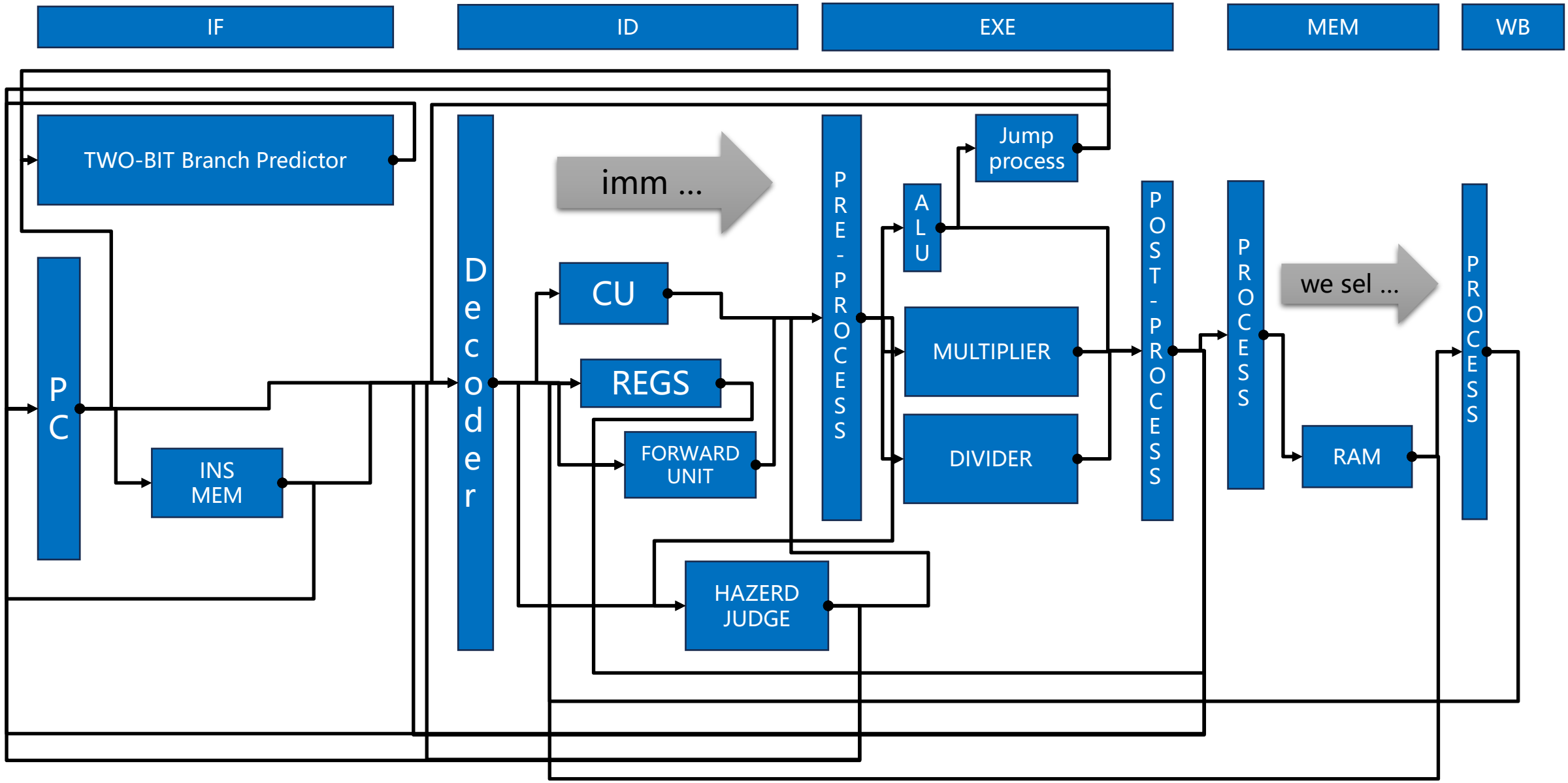
1 流水线设计

2 数据前推以及分支预测相关问题

3 上板演示

4 性能分析

流水线通路设计


IF
ID
EXE
MEM
WB
TWO-BIT Branch Predictor
PC
INS MEM
Decoder
imm ...
CU
REGS
FORWARD UNIT
HAZERD JUDGE
PRE-PROCESS
ALU
Jump process
MULTIPLIER
DIVIDER
POST-PROCESS
PROCESS
we sel ...
RAM
PROCESS

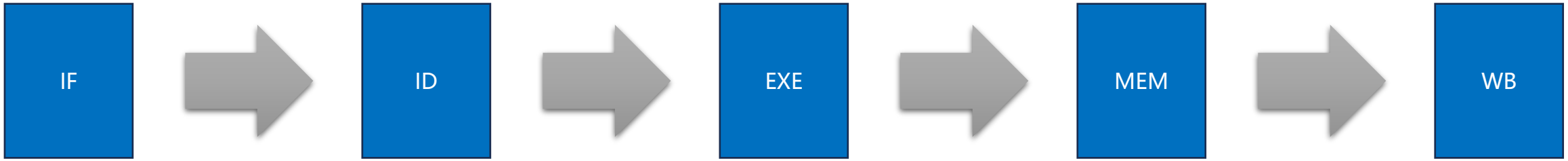
流水传递

前提：使用流水技术，每个阶段并行处理不同的指令

要求：每个阶段需接受前一阶段输入数据，实现该阶段功能，并将数据向下传递，保证流水正常

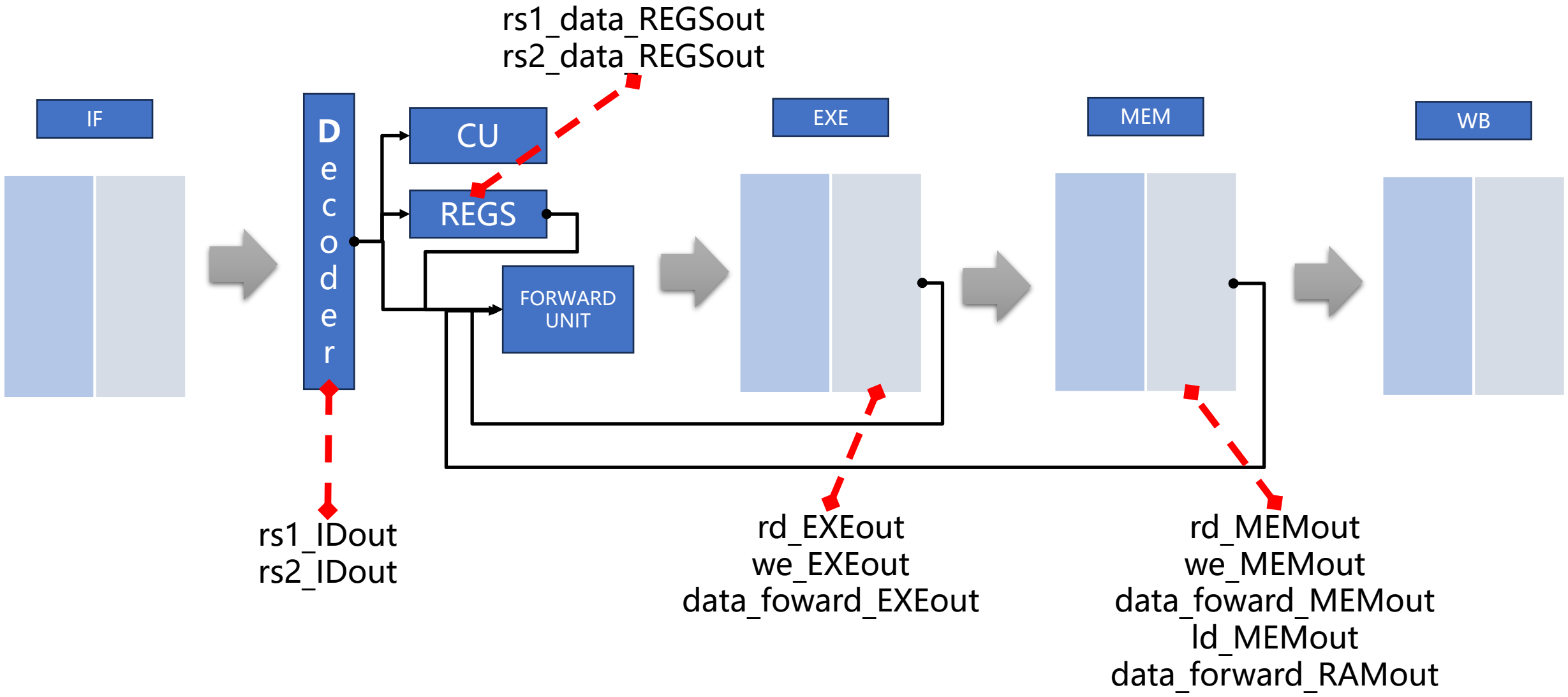
方法：上升沿将前一阶段准备好的数据做更新作为本阶段数据，同时将本阶段需要使用的数据做缓存

(缓存的原因为，前一阶段的线型输入数据在上升沿后会改变，因此需要将该阶段使用数据做缓存)

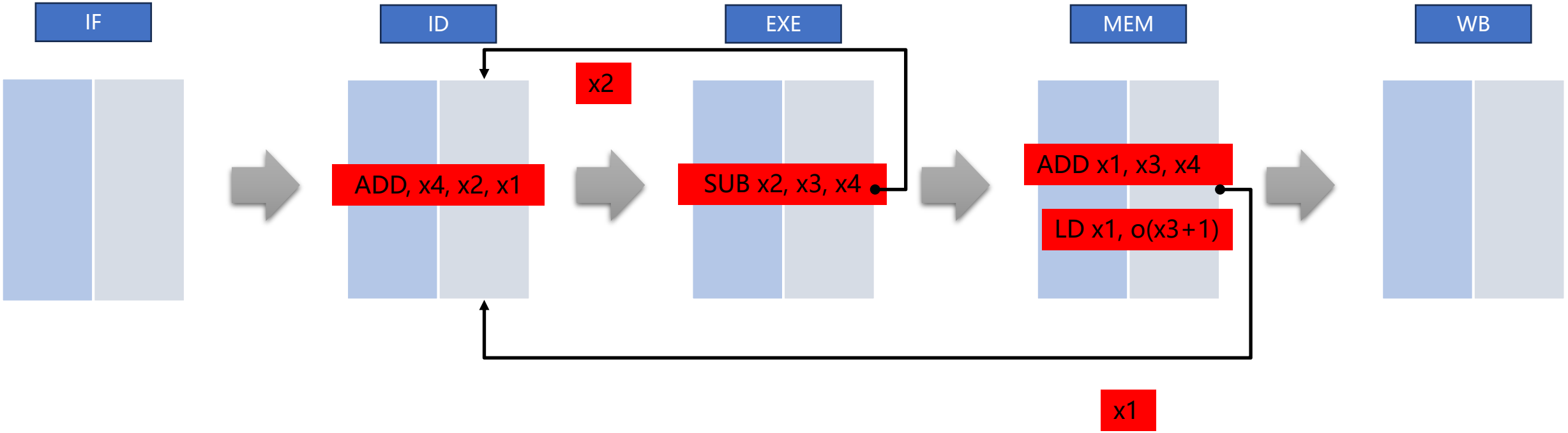


<取指>	<解码>	<执行>	<访存>	<回写>
PC	PC	使能信号	使能信号	回写使能
指令	使能信号	计算结果	计算结果	写入寄存器地址
	立即数	寄存器地址信息	内存数据	回写数据
	寄存器数	分支结果	写入寄存器地址	…
	…	…	…	

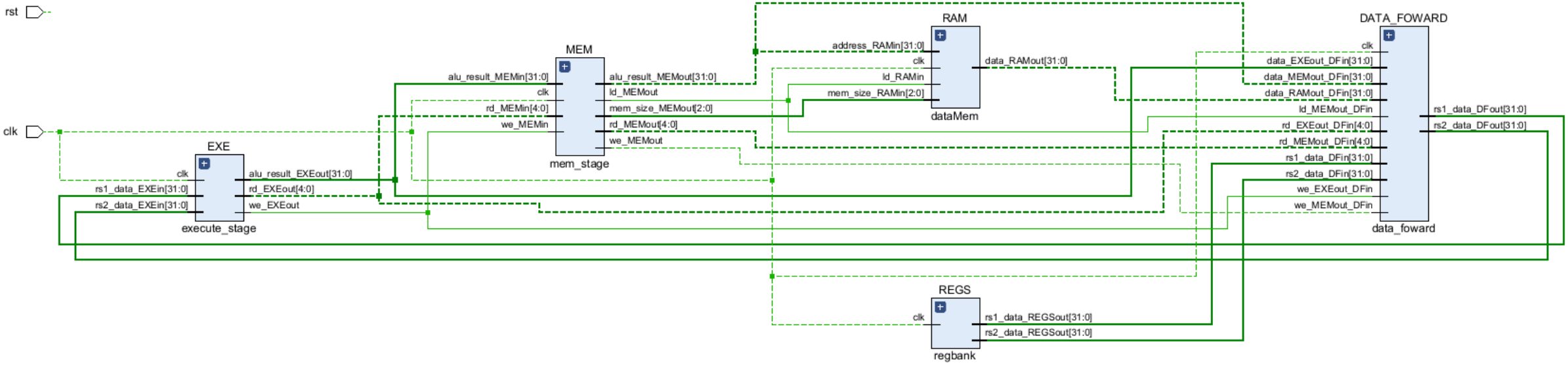
数据前推


rs1_data_REGSout
rs2_data_REGSout
IF
Decoder
CU
REGS
FORWARD UNIT
EXE
MEM
WB
rs1_IDout
rs2_IDout
rd_EXEout
we_EXEout
data_foward_EXEout
rd_MEMout
we_MEMout
data_foward_MEMout
ld_MEMout
data_forward_RAMout

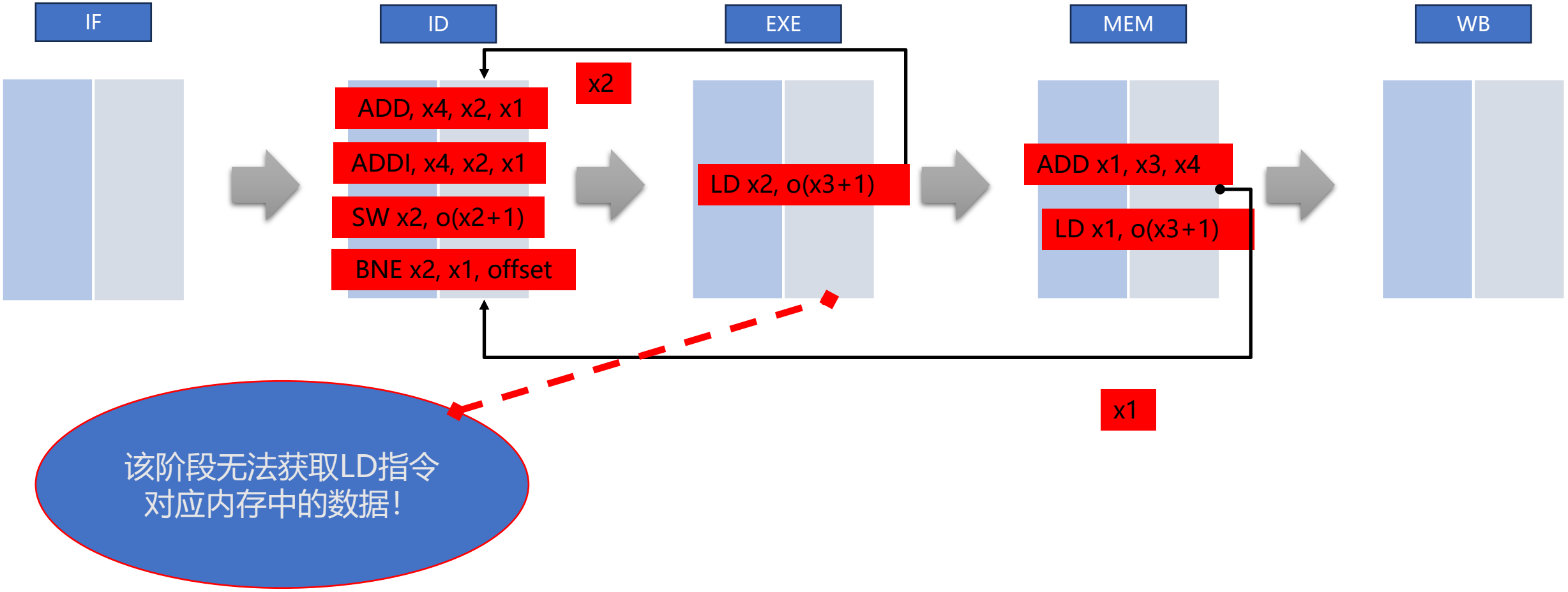
数据前推


IF
ID
EXE
MEM
WB
x2
ADD, x4, x2, x1
SUB x2, x3, x4
ADD x1, x3, x4
LD x1, o(x3+1)
x1

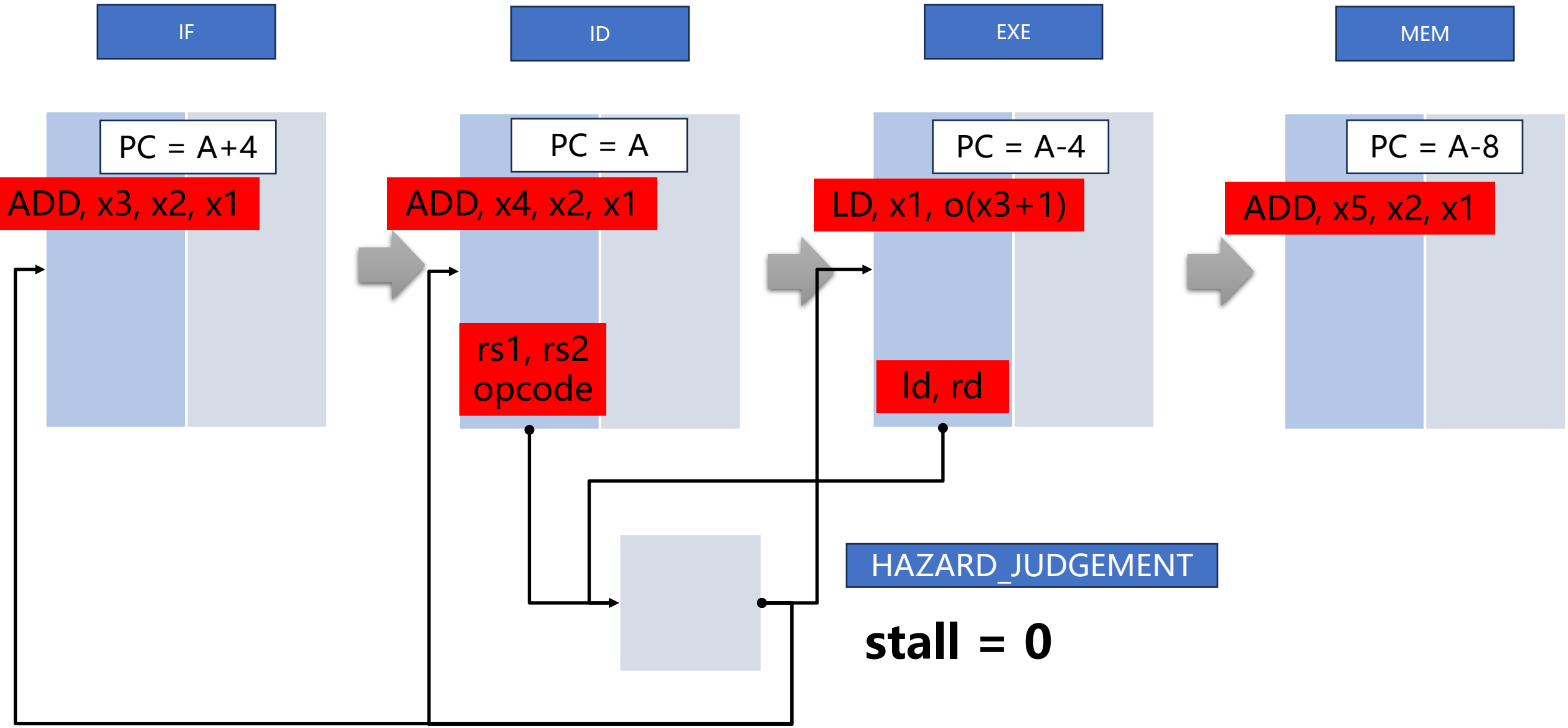
数据前推通路


rst
clk
EXE
clk
rs1_data_EXEin[31:0]
rs2_data_EXEin[31:0]
alu_result_EXEout[31:0]
rd_EXEout[4:0]
we_EXEout
execute_stage
MEM
alu_result_MEMin[31:0]
clk
rd_MEMin[4:0]
we_MEMin
alu_result_MEMout[31:0]
ld_MEMout
mem_size_MEMout[2:0]
rd_MEMout[4:0]
we_MEMout
mem_stage
RAM
address_RAMin[31:0]
clk
ld_RAMin
mem_size_RAMin[2:0]
data_RAMout[31:0]
dataMem
DATA_FOWARD
clk
data_EXEout_DFin[31:0]
data_MEMout_DFin[31:0]
data_RAMout_DFin[31:0]
ld_MEMout_DFin
rd_EXEout_DFin[4:0]
rd_MEMout_DFin[4:0]
rs1_data_DFin[31:0]
rs2_data_DFin[31:0]
we_EXEout_DFin
we_MEMout_DFin
rs1_data_DFout[31:0]
rs2_data_DFout[31:0]
data_foward
REGS
clk
rs1_data_REGSout[31:0]
rs2_data_REGSout[31:0]
regbank

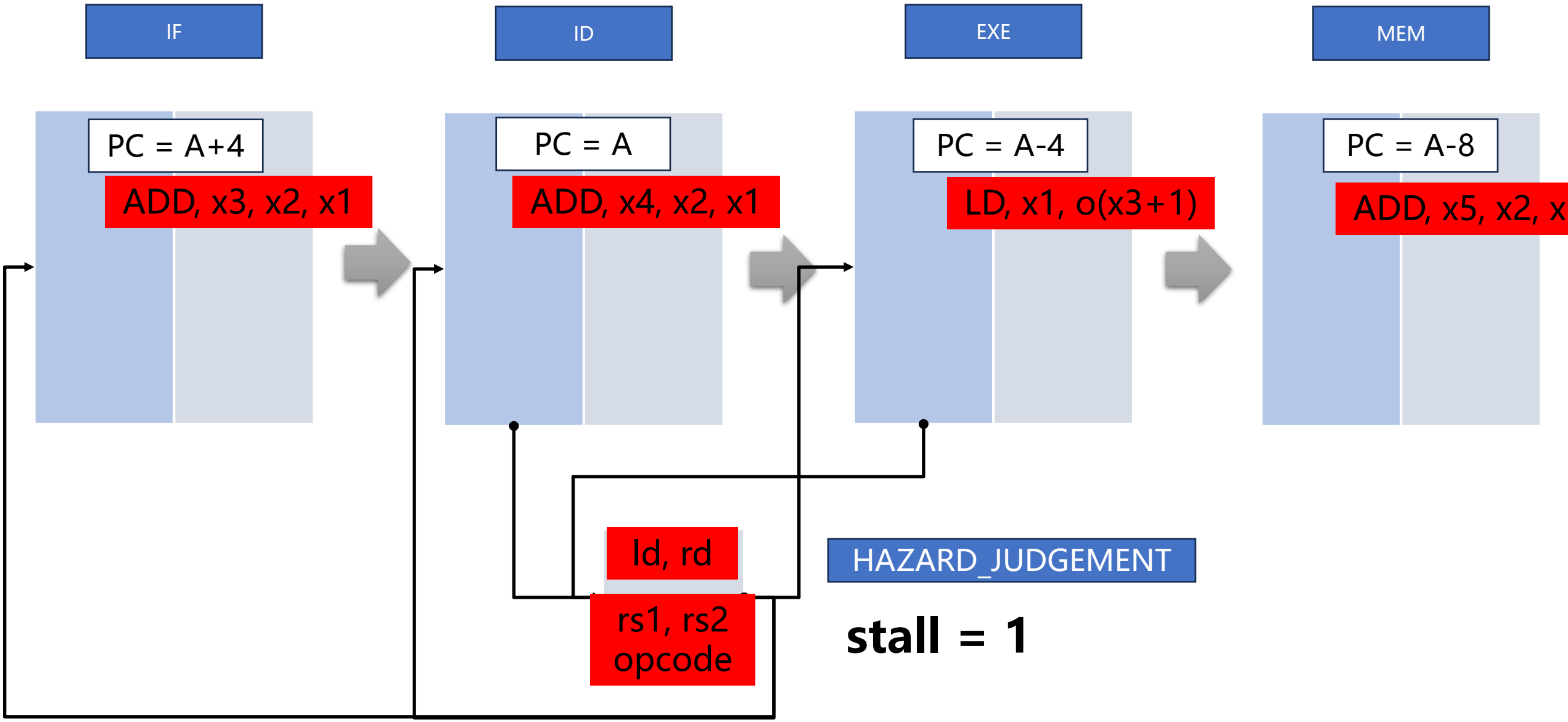
数据前推相关问题


IF
ID
EXE
MEM
WB
ADD, x4, x2, x1
ADDI, x4, x2, x1
SW x2, o(x2+1)
BNE x2, x1, offset
x2
LD x2, o(x3+1)
ADD x1, x3, x4
LD x1, o(x3+1)
x1
该阶段无法获取LD指令
对应内存中的数据！

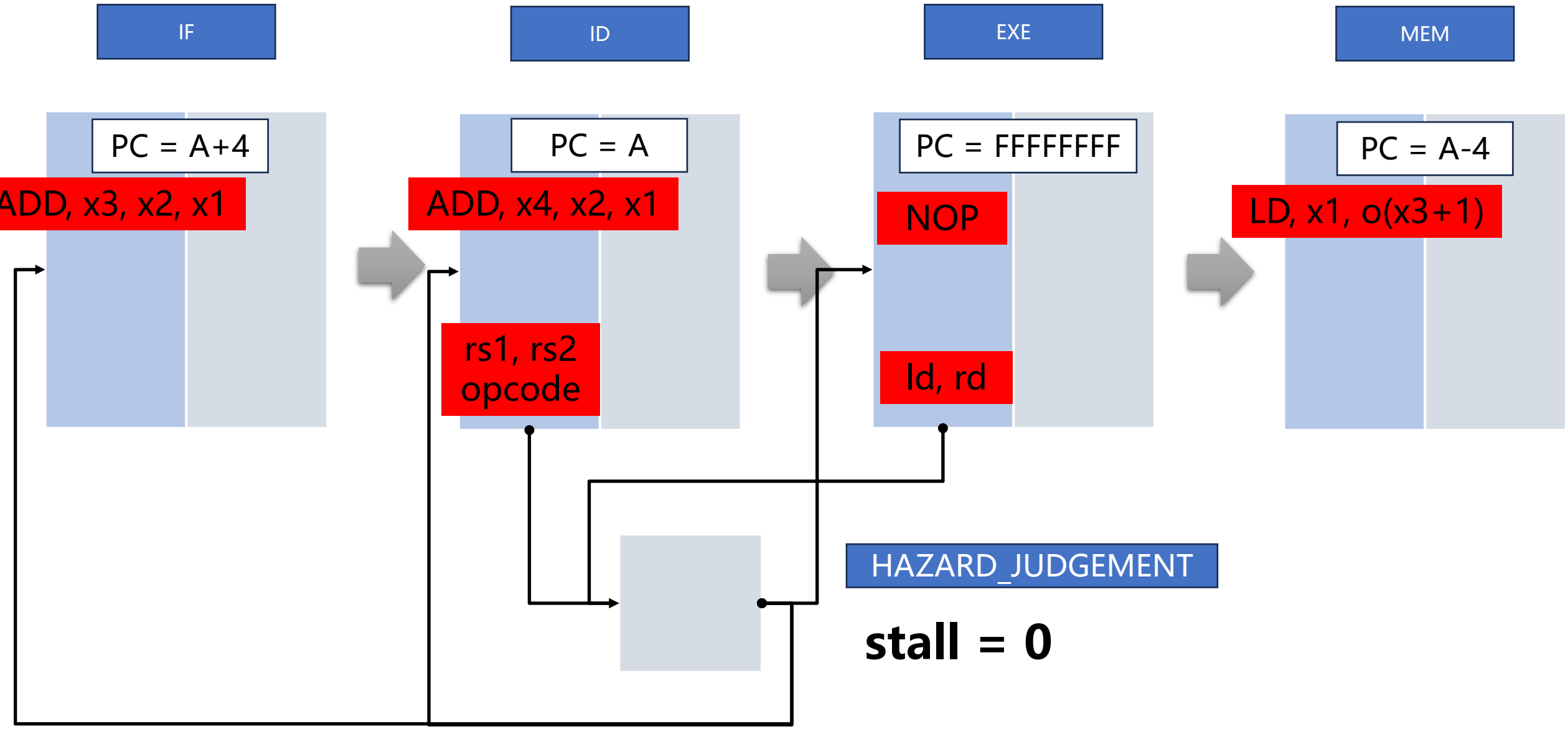
数据前推相关问题解决


IF
ID
EXE
MEM
PC = A+4
ADD, x3, x2, x1
PC = A
ADD, x4, x2, x1
rs1, rs2
opcode
PC = A-4
LD, x1, o(x3+1)
ld, rd
PC = A-8
ADD, x5, x2, x1
HAZARD_JUDGEMENT
stall = 0

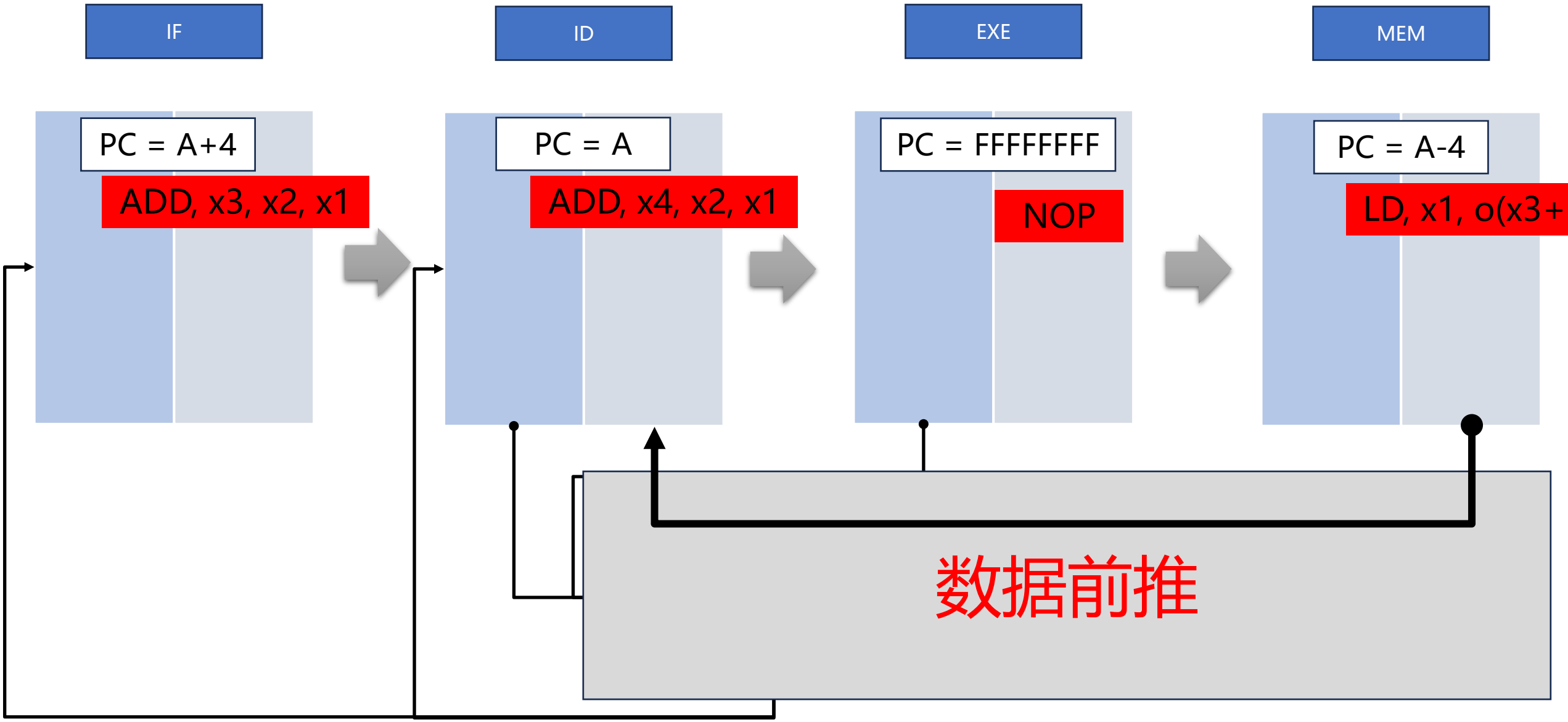
数据前推相关问题解决


IF
ID
EXE
MEM
PC = A+4
ADD, x3, x2, x1
PC = A
ADD, x4, x2, x1
PC = A-4
LD, x1, o(x3+1)
PC = A-8
ADD, x5, x2, x
ld, rd
rs1, rs2
opcode
HAZARD_JUDGEMENT
stall = 1

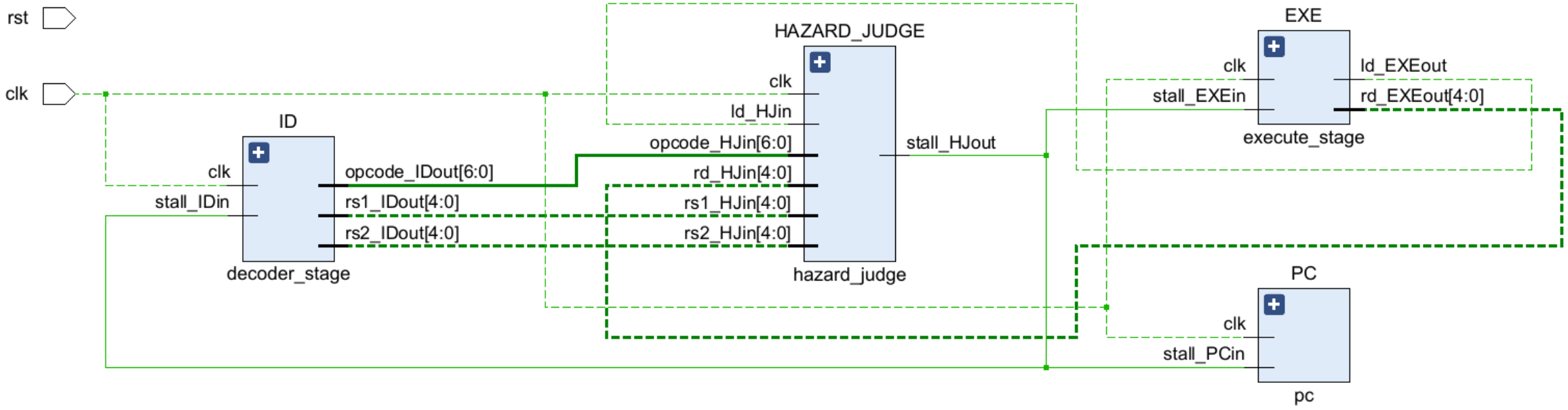
数据前推相关问题解决


IF
ID
EXE
MEM
PC = A+4
ADD, x3, x2, x1
PC = A
ADD, x4, x2, x1
rs1, rs2
opcode
PC = FFFFFFFF
NOP
Id, rd
PC = A-4
LD, x1, o(x3+1)
HAZARD_JUDGEMENT
stall = 0

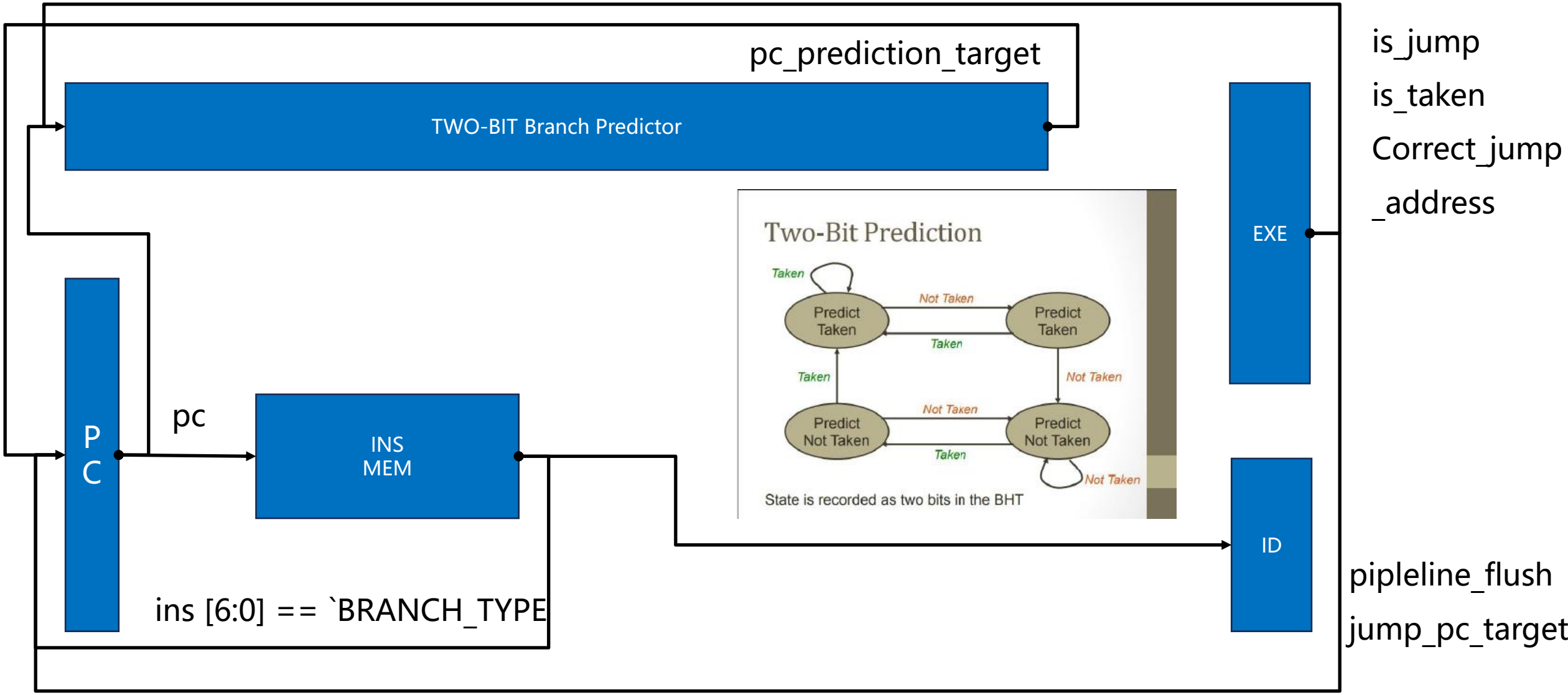
数据前推相关问题解决


IF
ID
EXE
MEM
PC = A+4
ADD, x3, x2, x1
PC = A
ADD, x4, x2, x1
PC = FFFFFFFF
NOP
PC = A-4
LD, x1, o(x3+
数据前推

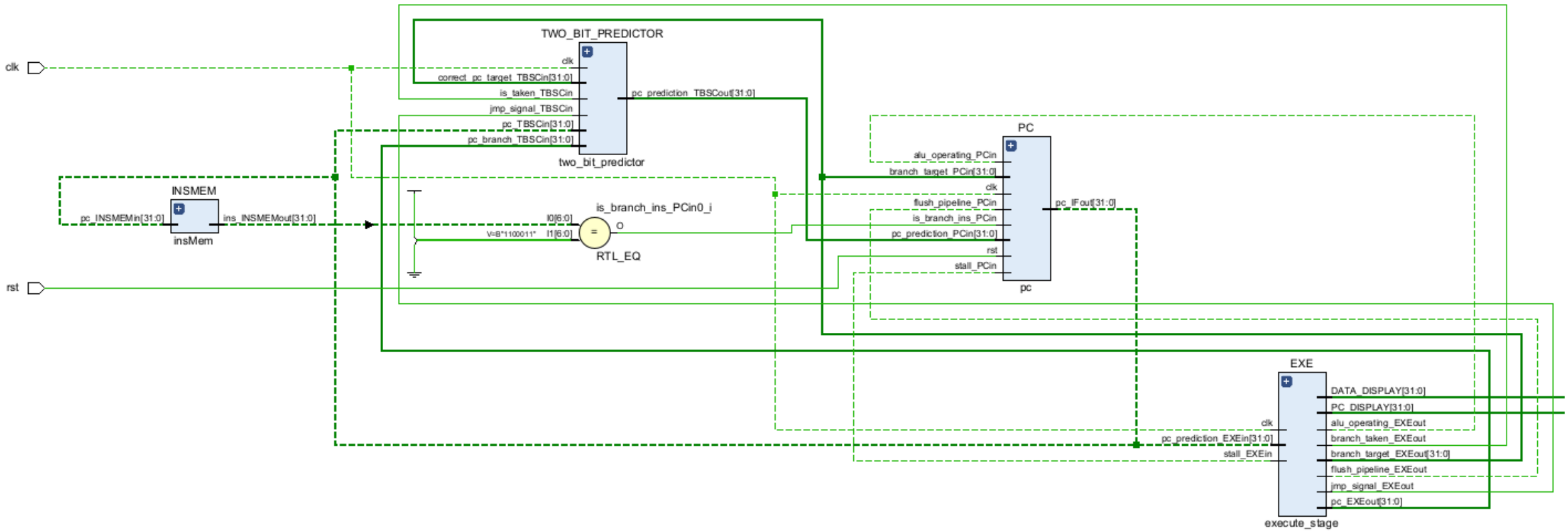
冲突检测


rst
clk
ID
clk
stall_IDin
opcode_IDout[6:0]
rs1_IDout[4:0]
rs2_IDout[4:0]
decoder_stage
HAZARD_JUDGE
clk
ld_HJin
opcode_HJin[6:0]
rd_HJin[4:0]
rs1_HJin[4:0]
rs2_HJin[4:0]
stall_HJout
hazard_judge
EXE
clk
stall_EXEin
ld_EXEout
rd_EXEout[4:0]
execute_stage
PC
clk
stall_PCin
pc

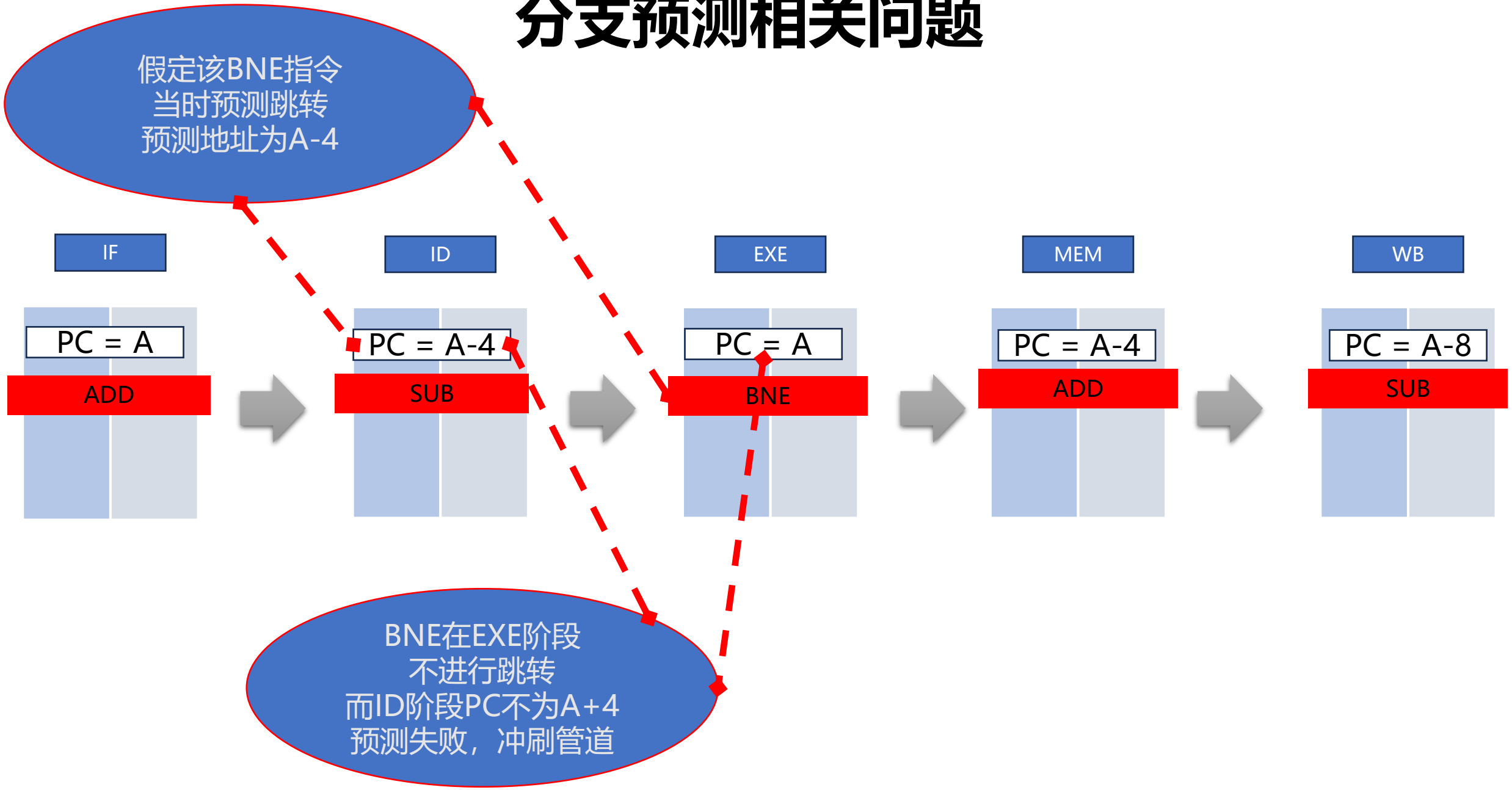
分支预测


pc_prediction_target
TWO-BIT Branch Predictor
is_jump
is_taken
Correct_jump
_address
EXE
Two-Bit Prediction
Taken
Not Taken
Predict Taken
Predict Taken
Taken
Taken
Not Taken
Not Taken
Predict Not Taken
Predict Not Taken
Taken
Not Taken
State is recorded as two bits in the BHT
P C
pc
INS MEM
ID
pipleline_flush
jump_pc_target
ins [6:0] == `BRANCH_TYPE

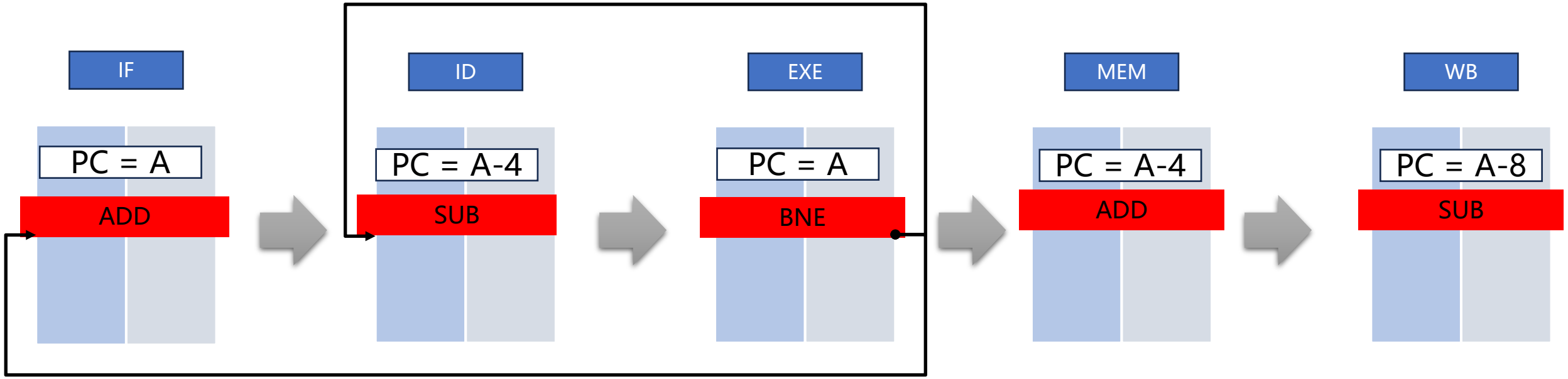
分支预测


clk
rst
TWO_BIT_PREDICTOR
clk
correct_pc_target_TBSCin[31:0]
is_taken_TBSCin
jmp_signal_TBSCin
pc_TBSCin[31:0]
pc_branch_TBSCin[31:0]
pc_prediction_TBSCout[31:0]
two_bit_predictor
INSMEM
pc_INSMEMin[31:0]
ins_INSMEMout[31:0]
insMem
is_branch_ins_PCin0_i
I0[6:0]
V=B"1100011"
I1[6:0]
O
RTL_EQ
PC
alu_operating_PCin
branch_target_PCin[31:0]
clk
flush_pipeline_PCin
is_branch_ins_PCin
pc_prediction_PCin[31:0]
rst
stall_PCin
pc_IFout[31:0]
pc
EXE
clk
pc_prediction_EXEin[31:0]
stall_EXEin
DATA_DISPLAY[31:0]
PC_DISPLAY[31:0]
alu_operating_EXEout
branch_taken_EXEout
branch_target_EXEout[31:0]
flush_pipeline_EXEout
jmp_signal_EXEout
pc_EXEout[31:0]
execute_stage

分支预测相关问题

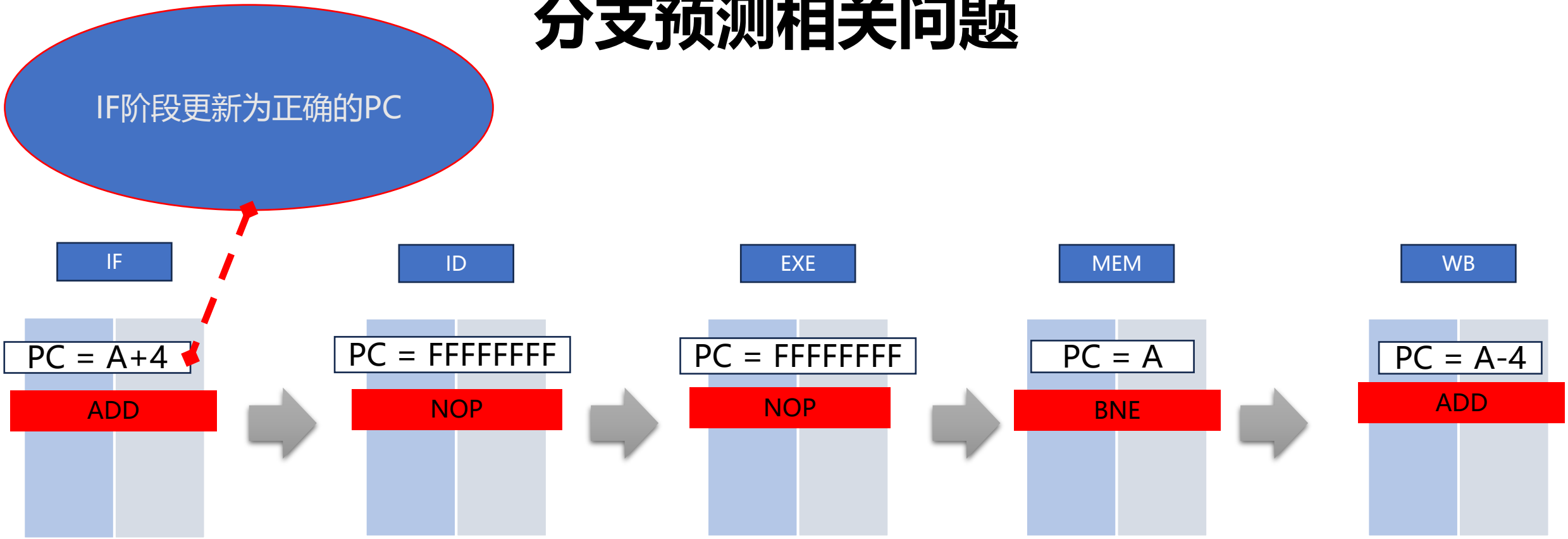

假定该BNE指令
当时预测跳转
预测地址为A-4
IF
ID
EXE
MEM
WB
PC = A
ADD
PC = A-4
SUB
PC = A
BNE
PC = A-4
ADD
PC = A-8
SUB
BNE在EXE阶段
不进行跳转
而ID阶段PC不为A+4
预测失败，冲刷管道

分支预测相关问题



pipeline_flush = 1

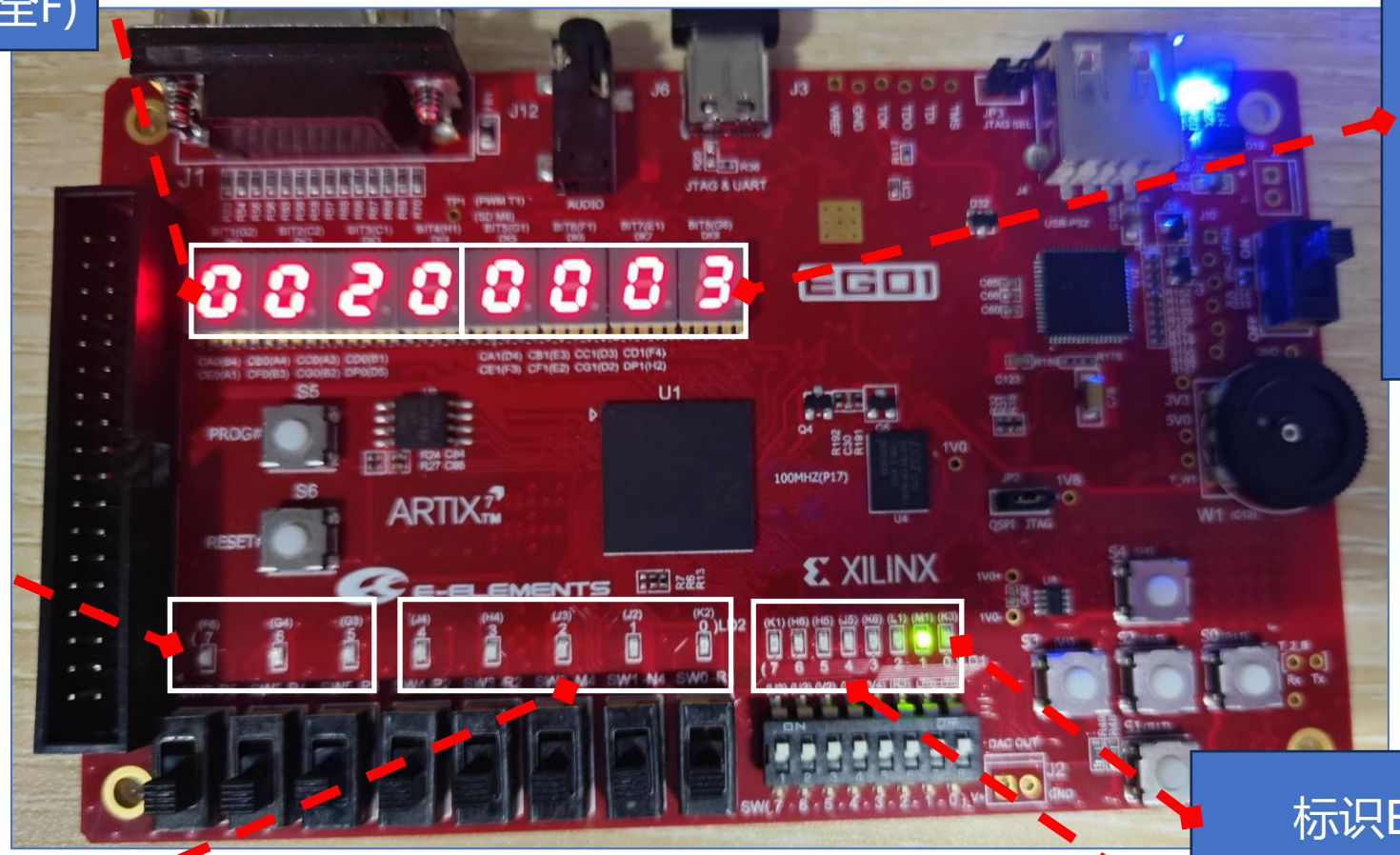
分支预测相关问题


IF阶段更新为正确的PC
IF
ID
EXE
MEM
WB
PC = A+4
ADD
PC = FFFFFFFF
NOP
PC = FFFFFFFF
NOP
PC = A
BNE
PC = A-4
ADD

板上信息相关

数码管前四位，十六进制
显示EXE阶段PC值低十六位
(如果是NOP指令显示为全F)

数码管后四位
显示EXE阶段计算信息
根据指令类型显示：
计算结果
PC正确跳转地址
储存数据
或访存地址
的低十六位



从左到右：
冲刷信号
冲突阻塞信号
ALU运转信号

标识EXE阶段指令类型

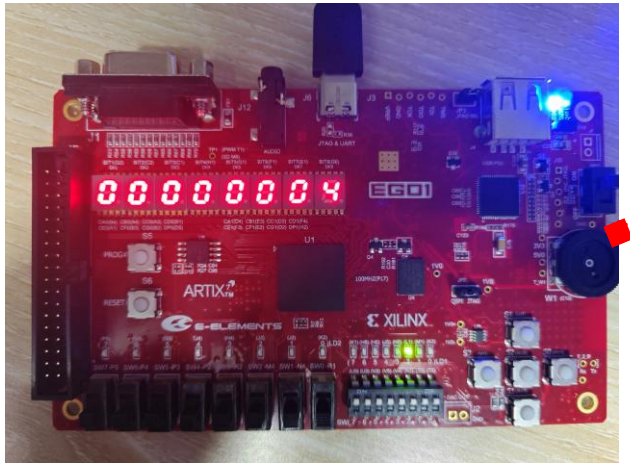
指示五个阶段空泡的流动
IF ID EXE MEM WB

7	6	5	4	3	2	1	0
NOP		JUMP	BRANCH	STORE	LOAD	IMM	LOGIC

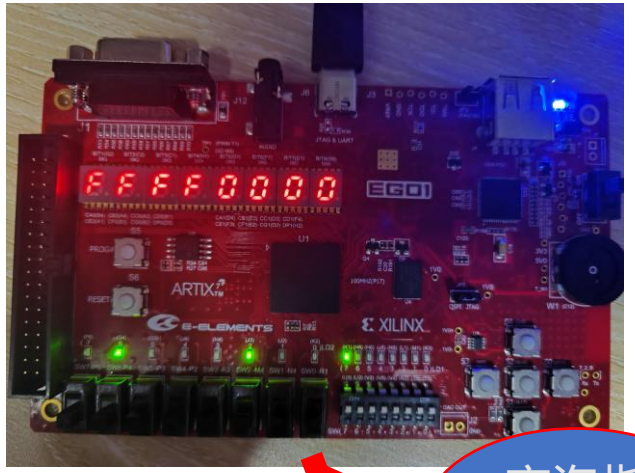
初始寄存器状态										
x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	…
00000000	00000001	00000002	00000004	00000008	xxxxxxxx	xxxxxxxx	xxxxxxxx	xxxxxxxx	xxxxxxxx	…

数据前推演示

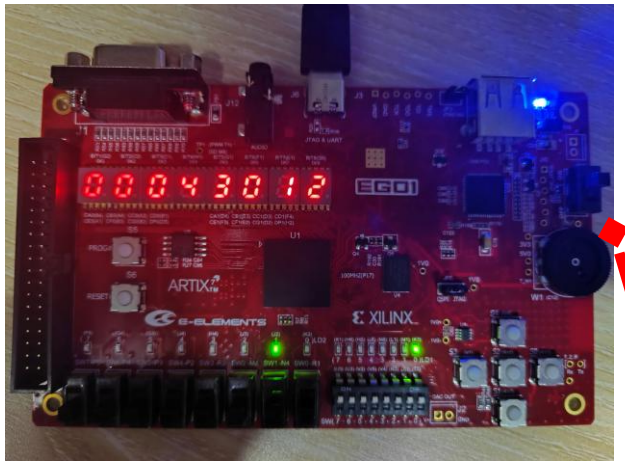
```
lw      x5,     o(x1+3)
add     x6,     x5,     x2
add     x7,     x1,     x2
add     x7,     x7,     x2
add     x8,     x7,     x2
add     x9,     x7,     x2
lw      x1,     o(x1+2)
andi    x7,     x2,     3
addi    x6,     x2,     3
ori     x5,     x2,     3
xori    x8,     x2,     3
lb      x1,     o(x2+2)
lh      x1,     o(x2+2)
xor     x4,     x3,     x2
or      x5,     x2,     x3
sw      x3,     o(x0+4)
sh      x3,     o(x0+4)
sb      x3,     o(x0+4)
```



从为4开始的地址块
取出一个字，
同时检测到冲突
插入空泡



空泡指
令显示



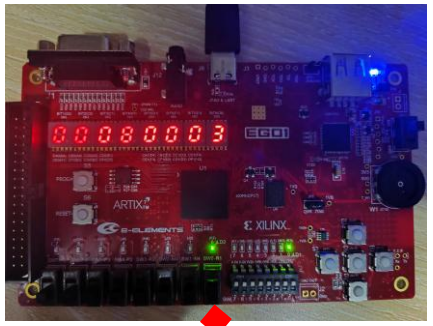
将访存地址数据前
推参与逻辑运算

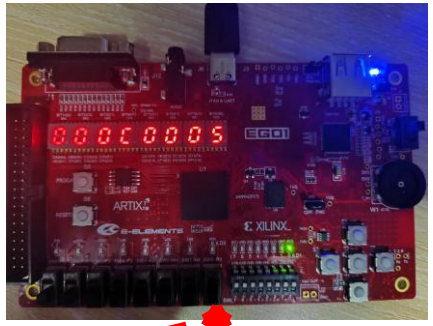
RAM	
0000	0F
0001	07
0002	03
0003	01
0004	F0
0005	70
0006	30
0007	10
0008	00
0009	00
000A	00
000B	01
…	…

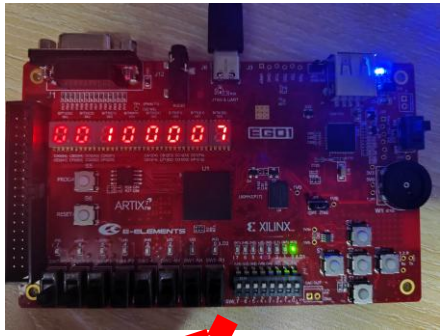
初始寄存器状态										
x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	…
00000000	00000001	00000002	00000004	00000008	F0703010	F0703012	xxxxxxxx	xxxxxxxx	xxxxxxxx	…

数据前推演示

```
lw      x5,     o(x1+3)
add     x6,     x5,     x2
add     x7,     x1,     x2
add     x7,     x7,     x2
add     x8,     x7,     x2
add     x9,     x7,     x2
lw      x1,     o(x1+2)
andi    x7,     x2,     3
addi    x6,     x2,     3
ori     x5,     x2,     3
xori    x8,     x2,     3
lb      x1,     o(x2+2)
lh      x1,     o(x2+2)
xor     x4,     x3,     x2
or      x5,     x2,     x3
sw      x3,     o(x0+4)
sh      x3,     o(x0+4)
sb      x3,     o(x0+4)
```

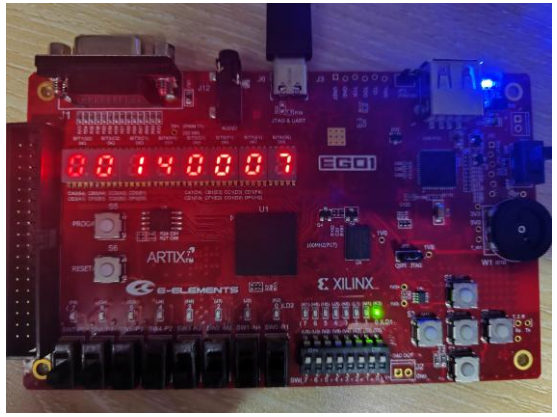






将EXE阶段计算结果前推参与计算

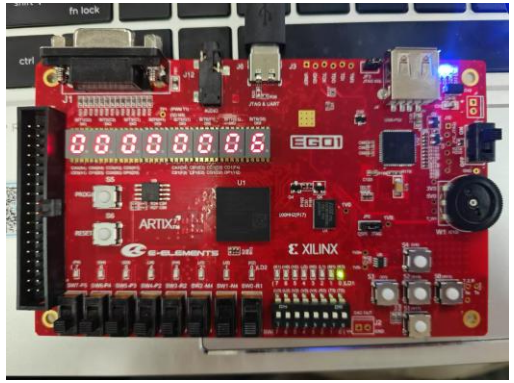
将同一写入寄存器最新状态前推参与计算

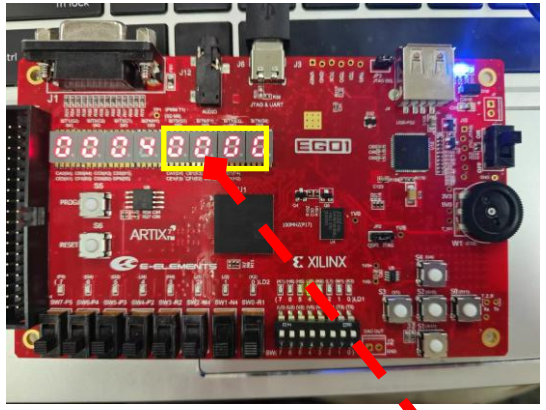


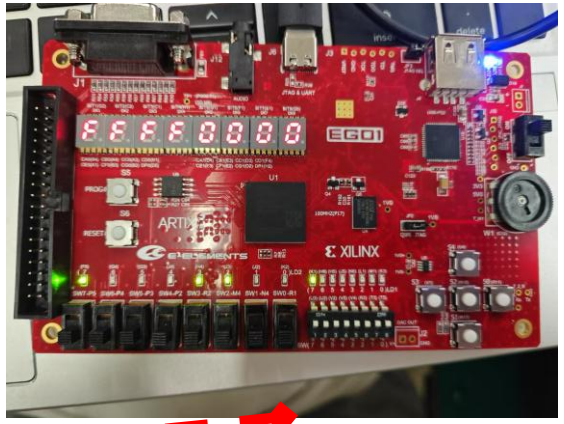
RAM	
0000	0F
0001	07
0002	03
0003	01
0004	F0
0005	70
0006	30
0007	10
0008	00
0009	00
000A	00
000B	01
…	…

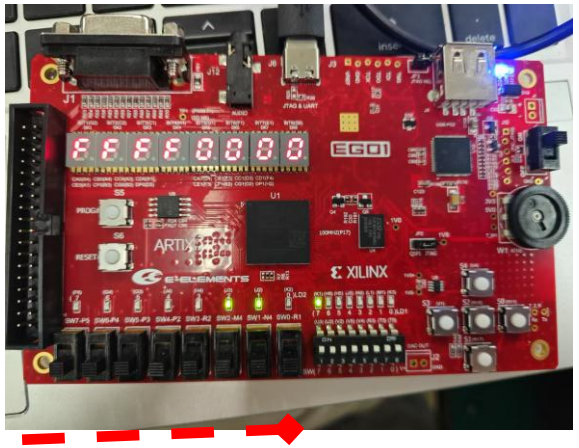
```
add     x7,     x2,     x3
bge     x4,     x2,     offset(-2)
add     x7,     x2,     x3
add     x7,     x2,     x3
```

分支预测失败处理





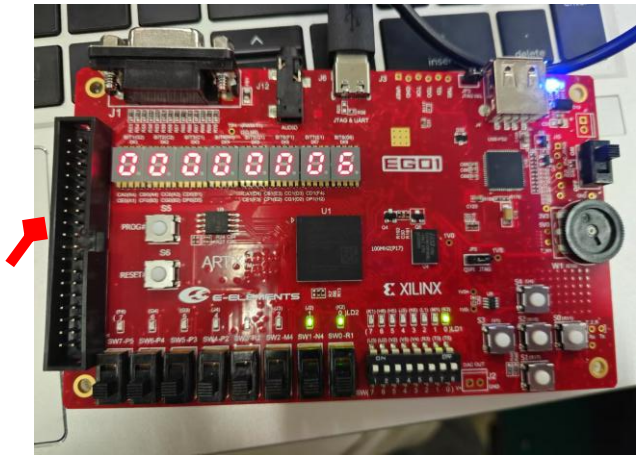


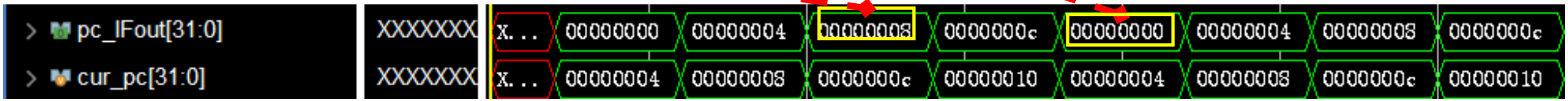


正确地址为0，预测失败，插入两个空泡，以及流动过程

分支指令，根据预测器初始状态强不跳转，取下一条指令

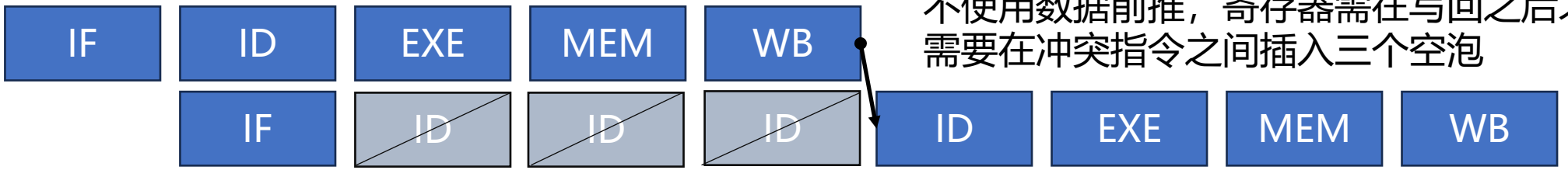
正确指令流入



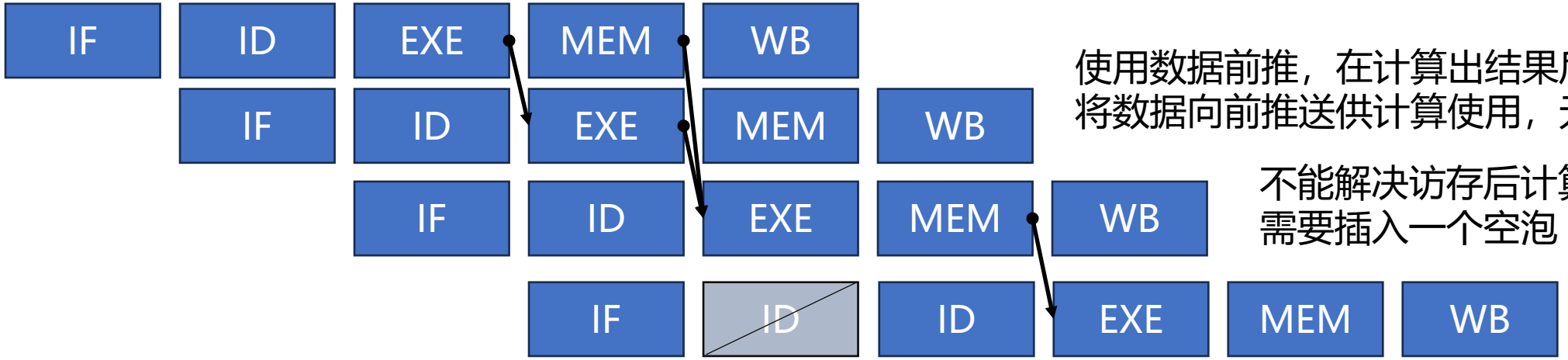


性能优化分析

使用数据前推可以使得本应在WB阶段之后才能访问的寄存器数据能够在更早的时间内访问，
不过无法仍旧不能完美解决一些特殊的指令序列带来的冲突



不使用数据前推，寄存器需在写回之后才能访问
需要在冲突指令之间插入三个空泡



使用数据前推，在计算出结果后或者访存后可以
将数据向前推送供计算使用，无需插入空泡

不能解决访存后计算使用的情况
需要插入一个空泡

性能优化分析

不使用数据前推，冲突指令序列之间插入三个空泡

```
lw      x5,     o(x1+3)
add     x6,     x5,     x2
add     x7,     x1,     x2
add     x7,     x7,     x2
add     x8,     x7,     x2
add     x9,     x7,     x2
lw      x1,     o(x1+2)
andi    x7,     x2,     3
addi    x6,     x2,     3
ori     x5,     x2,     3
xori    x8,     x2,     3
lb      x1,     o(x2+2)
lh      x1,     o(x2+2)
xor     x4,     x3,     x2
or      x5,     x2,     x3
sw      x3,     o(x0+4)
sh      x3,     o(x0+4)
sb      x3,     o(x0+4)
```

3 stalls

3 stalls

3 stalls

使用数据前推，访存后冲突指令序列之间插入一个空泡

```
lw      x5,     o(x1+3)
add     x6,     x5,     x2
add     x7,     x1,     x2
add     x7,     x7,     x2
add     x8,     x7,     x2
add     x9,     x7,     x2
lw      x1,     o(x1+2)
andi    x7,     x2,     3
addi    x6,     x2,     3
ori     x5,     x2,     3
xori    x8,     x2,     3
lb      x1,     o(x2+2)
lh      x1,     o(x2+2)
xor     x4,     x3,     x2
or      x5,     x2,     x3
sw      x3,     o(x0+4)
sh      x3,     o(x0+4)
sb      x3,     o(x0+4)
```

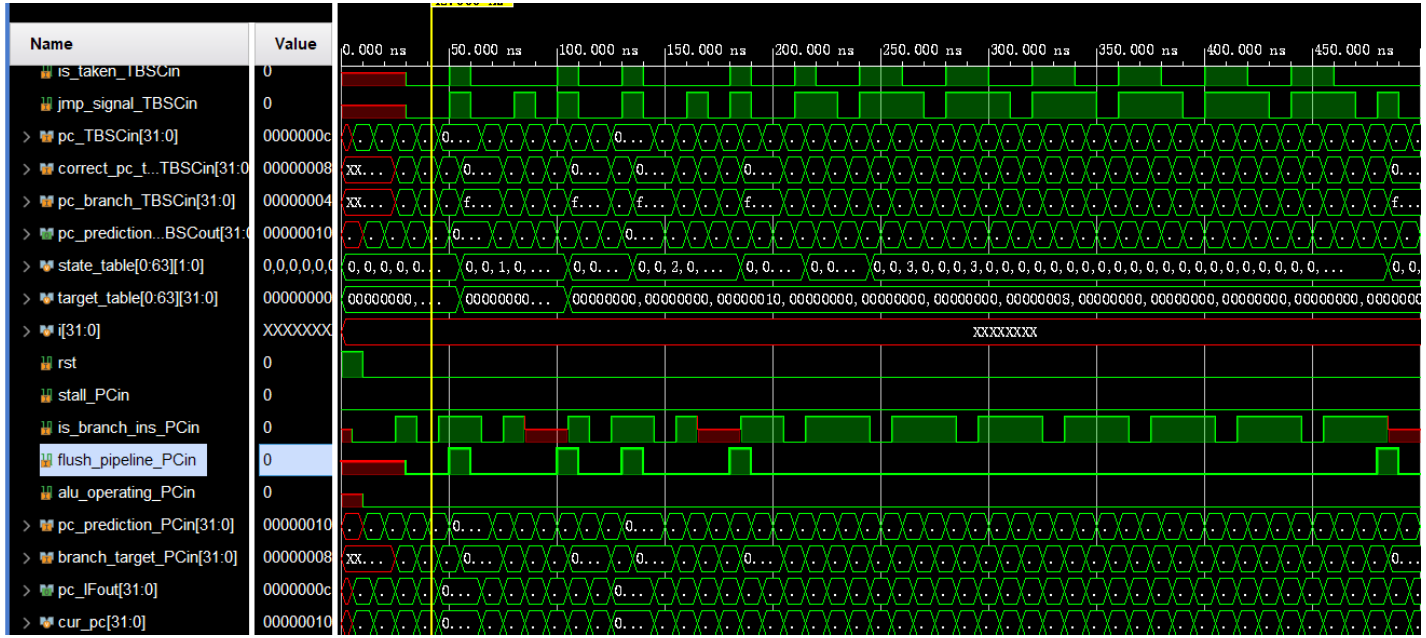
1 stalls

分支预测性能优化分析

一个9层循环中有两条分支指令一个跳转一个不跳转，共冲刷5次流水线。27(2*9+9)次分支判断

分支预测，2-bit预测器以及全局地址跳转表
对过往存在的跳转分支进行表项存储
每一项包含一个2bit位以及跳转地址
设置表大小为64，选取pc[7:2]位作为索引值

```
add     x2,     x0,     x0
add     x3,     x0,     x0
bge     x1,     x1,     offset(1)
add     x2,     x1,     x2
bne     x1,     x1,     offset(1)
add     x3,     x1,     x3
bge     x4,     x3,     offset(-5)
```



预测方式	预测成功率
2bit	81%
静态跳转	62%
静态不跳转	37%

谢谢观看